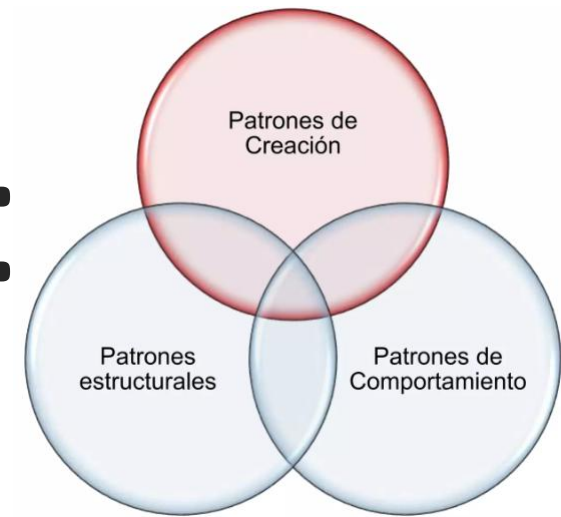


PDS-CREACIONALES: FACTORY METHOD



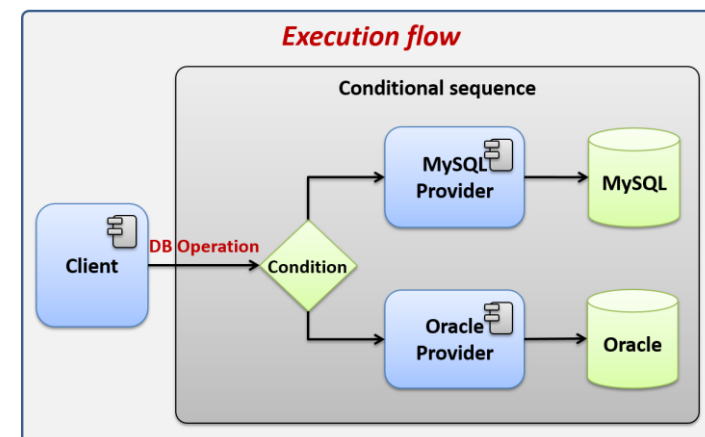
ING. ZOILA RUIZ, PHD

FACTORY METHOD

Es un patrón de diseño creacional, orientados a cómo se crean los objetos en la aplicación, que proporciona una interfaz para crear objetos (superclase), mientras permite a las subclases alterar el tipo de objetos que se crearán/instanciar.

Su objetivo es centralizar el sitio donde se crean los objetos, normalmente donde se crean objetos de una misma familia o tipo, como podrían ser componentes visuales, de lógica del negocio, u objetos concurrentes en el tiempo.

En lugar de instanciar objetos directamente con new, el cliente llama a un **método fábrica** que devuelve una instancia del objeto específico.



FACTORY METHOD

Factory Method

- ❖ Este método permite la creación de objetos de un subtipo determinado a través de una clase Factory.
- ❖ El Factory encapsula la lógica de creación de los objetos, de tal forma que el resultado esta encapsulado tras una interfaz/clase común, desacoplando al cliente de la clase del objeto creado

Es una solución elegante para separar la lógica de creación de objetos del resto del código. Pese a que añade un poco de complejidad, esta se compensa con un diseño más limpio y mantenible.

FACTORY METHOD

Contexto: El código cliente necesita crear objetos, pero no debe depender de las clases concretas.

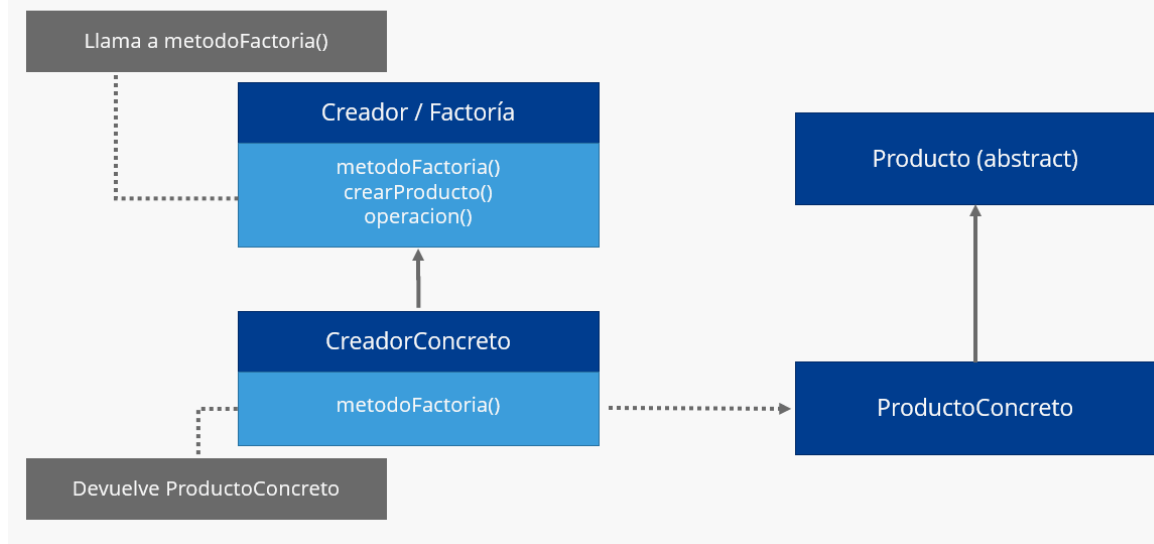
Problema: ¿Cómo permitir que las subclases decidan qué clase concreta instanciar sin modificar el código cliente?

Solución: Definir un método de creación abstracto (factoryMethod) en una clase base, y dejar que las subclases lo implementen para crear objetos específicos.

CONSECUENCIAS POSITIVAS	CONSECUENCIAS NEGATIVAS
Flexibilidad en la creación de objetos	Aumenta el número de clases
Facilita el principio abierto/cerrado	Puede complejizar el diseño si se abusa
Sigue el principio de responsabilidad única	

FACTORY METHOD

Diagrama UML: patrón de diseño Factory



Para la implementación del patrón se debe considerar los siguientes puntos:

Todos los productos a instanciar deben seguir una misma interfaz o clase abstracta(superclase).

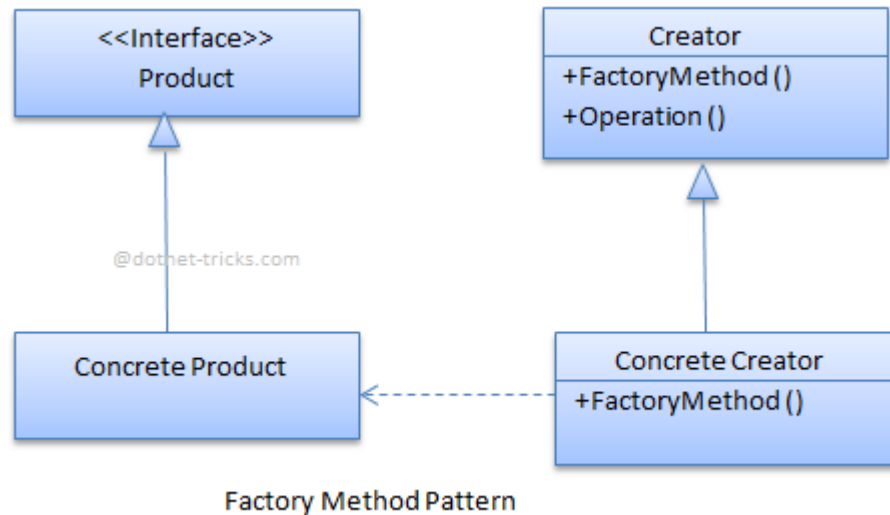
La interfaz/clase abstracta debe declarar métodos que sean generales a todos los productos.

La clase encargada de la instanciación de objetos debe declarar un método que retorne un tipo interfaz/clase común a los productos.

Debe encontrar todas las referencias a constructores de producto en el código de la clases creadora.

FACTORY METHOD

Diagrama UML: patrón Factory Method



La arquitectura del patrón tiene los siguientes elementos:

Product: es la interfaz/clase abstracta común a todos los objetos que se pueden crear en la clase **Creator** y sus subclases.

ConcretProduct: clase que implementa o extiende la interfaz **Product**.

Creator: clase abs/interfaz que declara el método fabrica (FM), que crea y retorna nuevos **Product** (objetos).

ConcreteCreator: Es la clase **Factory** que implementa la lógica del patron.

FACTORY METHOD

Usos:

Cuando la creación directa de un objeto por medio del operador new resulta perjudicial.

Cuando no se conoce en tiempo de diseño, la subclase que se utilizará.

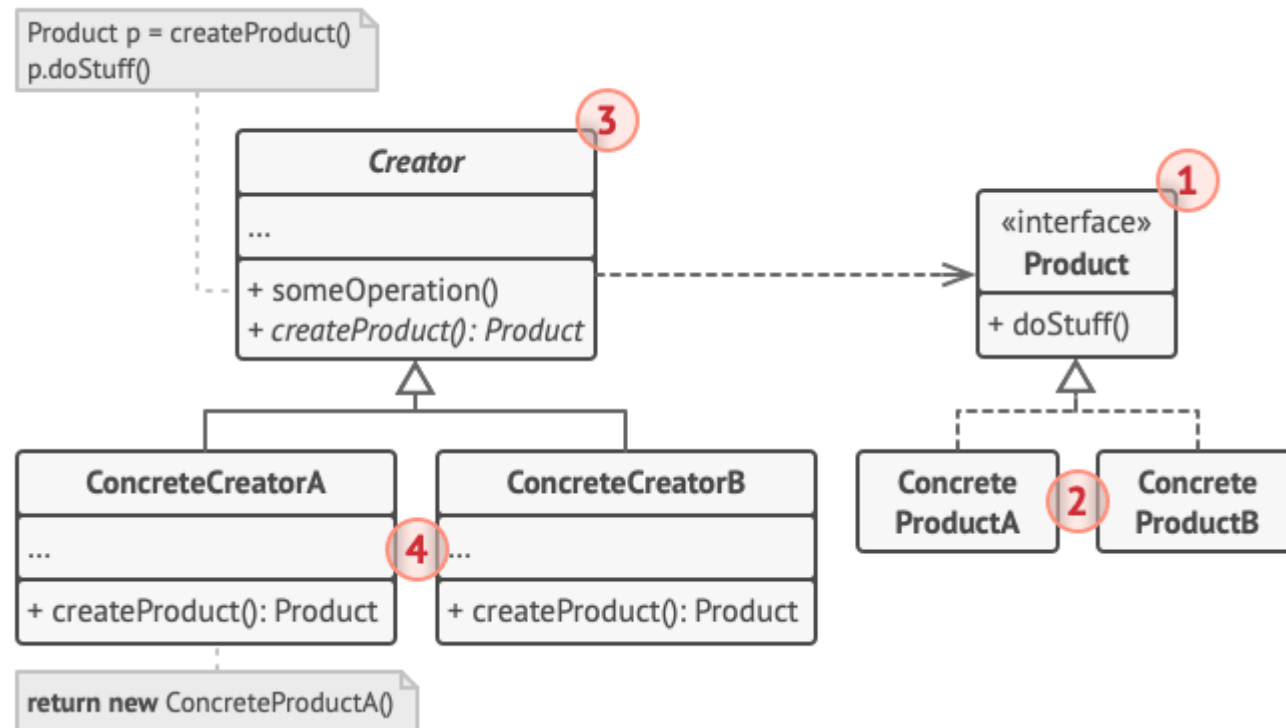
Cuando construimos un objeto basado en una serie de condiciones else if o switch

Cuando deseas centralizar la creación de objetos.

Cuando el código cliente no debería depender de clases concretas.

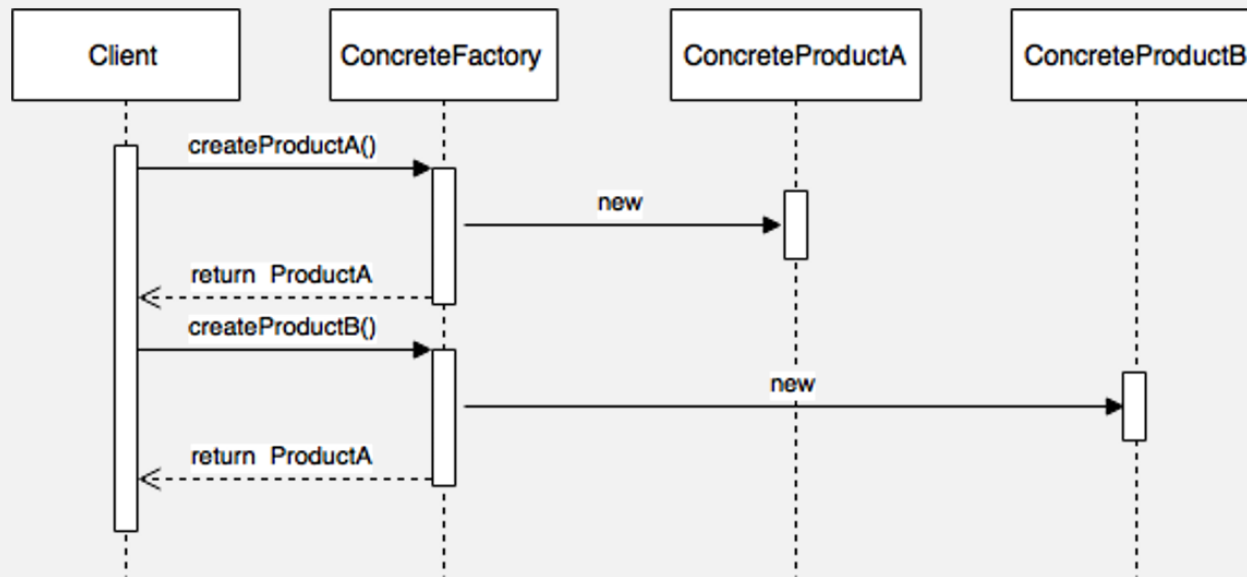
Cuando la lógica de creación es compleja (por ejemplo, depende de datos de configuración).

FACTORY METHOD



FACTORY METHOD

Factory Method – Diagram of sequence



- ❖ Permite a través de un punto central (método factory) en tiempo de ejecución decidir que subclase instanciar.
- ❖ Permite incorporar nuevos productos sin afectar la lógica de creación.